

introduction C programming language

error

- Semantic error
- Syntax error

Debugger

bug detection

editor

Compiler

Debugger

} IDE
integrated
Development
environment

System programming language

Unix Os

1972

Mid-level programming language

Case-Sensitive

Compiler (Derbyci)

Source file(s) (text)

Object file(s) (binary)

Linker

Executable File (Binary)

IDEs

Dev C++

Visual Studio

Eclipse

Notepad++

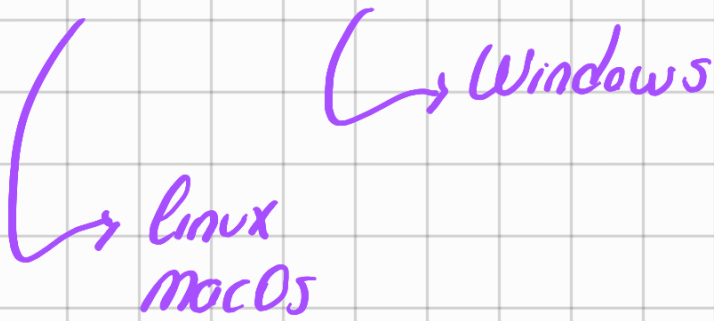
Code Blocks

Clion

toploma.c (source file)

toploma.o (object file)

toploma / toploma.exe (executable File)



```
#include <stdio.h>      "stdio.h"
```

```
int main() {
```

```
    return 0;  —————> exit(0);
```

```
}
```

Read / write operations

```
scanf("%d", &0);
```

gets()

(—————> Variable

→ *ampersand*
puts()

```
printf("%d", a);
```

```
printf("Hello world");
```

```
printf("Hello world\n");
```

```
scanf("%d %d", &a, &b);
```

```
scanf("%d", &a);
```

```
scanf("%d", &b);
```

```
printf("A'nin degeri : %d - B'nin degeri  
%d", a, b);
```

Aritmetic operations

+ Addition

- Subtraction

* Multiplication

/ Division

% modular arithmetic

5/2 → 2 Div

5.0 / 2
5 / 2.0
5.0 / 2.0

} 2.5

Control flow (if)

if(x) → non-zero (true)
→ zero (false)

Statement 1,

Statement 2,

if(1) {
if(5) {
if(-5) {

} true

if(0) → false

if(x) {

Statement 1;

}
else {

Statement 2;

}

Statement 3;

if(x) {

Statement 1;

Statement 2;

}

else {

Statement 3;

Statement 4;

}

Statement 5;

Comparison expressions

< less than

> greater than

<= less than or equal to

>= greater than or equal to

== equal to

$\neq$ not equal to

```
if  $-1 < 0$  {  
    1  
}  
else {  
  
}
```

```
if  $0 < -1$  {  
  
}  
else {  
    1  
}
```

```
if  $-1 < 0 < 1$  {  
    1 < 1  
    0  
}  
else {  
    1  
}
```

```
if  $-1 < 0 < 2$  {  
    1 < 2  
}
```

$\underbrace{1, 2}$
}
else {
}

logical operations

And &&
Or ||

$x < y < z$

if (($x < y$) && ($y < z$))

if (($x < 0$) || ($x > 100$))

Nested if statements

```
if (a < b) {  
    if (a < c) {  
        d = a;  
    }  
    else {  
        d = c;  
    }  
}
```

a, b, c
3 4 5


```
else if (b < c) {  
    d = b;  
}  
else {  
    d = c;  
}  
printf("%d", d);
```

Scalar Data types

Char (1 byte)
int
float
double

Short unsigned
long signed

char < short int ≤ int ≤ long int

float < double

Short int (2 byte)
Long int 4-8 byte
int 4 byte

unsigned int a;

Variable declaration

int a, b, c;

float x; x = 0.0;

double y, z;

char harf, m;

float x = 0.0;

int main() {

int num1, num2, result;

printf("iki sayi giriniz \n");

scanf("%d", &num1);

scanf("%d", &num2);

result = num1 + num2;

printf("Sonuç = %d", result);

return 0;

}

↳ iki sayı giriniz

5 (enter)

6 (enter)

5 6 = 11

loops

for
while
do while

for statement

for($i=0$; $i < 100$; $i++$) {
 $\swarrow$ exp1 $\searrow$ exp2 $\rightarrow$ exp3

statement 1;

$i = i + 2$ $i = i * 2$

INC	ADD
$i++$	$i = i + 1$
Dec	sub
$i--$	$i = i - 1$

Do-while statement

$i = 0$

do {

i=i+1

statement1;

} while (i < 100);

While statement

i = 0

while (i < 100) {

statement1;

i++;

}

scanf("%d %d", &x, &y)

while (x < y) {

x++;

y--;

}

Arrays

`int a[100];` → `a[0]` ——— `a[99]`

`float x[50];` → `x[0]` ——— `x[49]`

`scanf("%d", &a[N]);`

`for(i=0; i<N; i++) {`

`int a[N];`
`scanf("%d", &a[i]);`

`}`

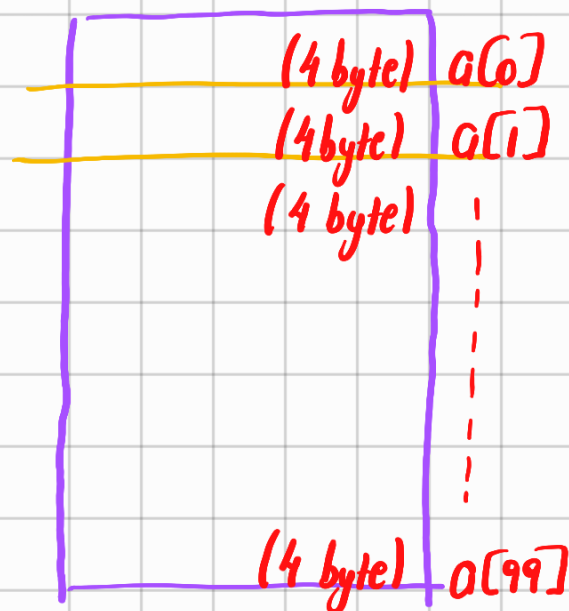
`for(i=0; i<N; i++) {`

`printf("%f", x[i]);`

`a[100]` } segment
`x[50]` } location
`x[100]` } fault

`#define N 100`

`int a[N];`



C programming language

- general structure

read/write operation

Control flow (if)

Arithmetic operations, Comparison expressions

Scalar data type

loops (for, while, do-while)

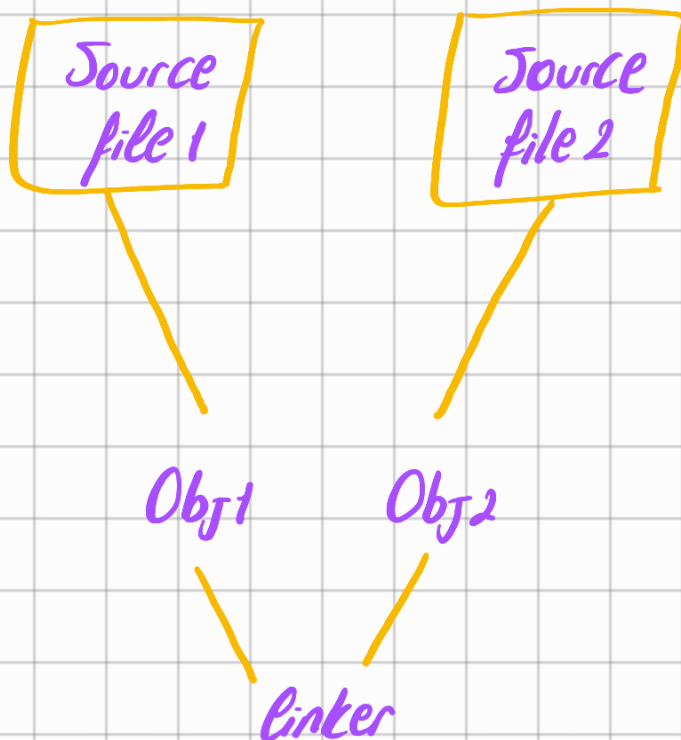
Arrays

Multi-dimensional Arrays

Array of characters (string)

```
int main() {  
    // comment line  
  
    /* block 1  
       block 2  
  
    */  
    return 0;  
}
```

Ansi C-standart



Read / write operations

decimal %d

character %c

floating-point %f

String %s

hexadecimal %x

1	2	-3	5	0	7	6
---	---	----	---	---	---	---

Void type

'a'	'b'	'i'	'j'	''	'j'	'j'	''
-----	-----	-----	-----	----	-----	-----	----

1.1	2.1	3.2
-----	-----	-----

Char a;

```
scanf("%c", &a);
```

```
printf("%c", a);
```

```
printf("%d", a);
```